Architecture 1001: x86-64 Assembly SpecialMaths.c

Xeno Kovah – 2021

xeno@darkmentor.com

All materials is licensed under a Creative Commons “Share Alike” license.

- http://creativecommons.org/licenses/by-sa/4.0/

You are free:

to Share — to copy, distribute and transmit the work

to Remix — to adapt the work

Under the following conditions:

Attribution — You must attribute the work in the manner specified by the author or licensor (but not in any way that suggests that they endorse you or your use of the work).

Share Alike — If you alter, transform, or build upon this work, you may distribute the resulting work only under the same, similar or a compatible license.

Attribution condition: You must indicate that derivative work
"Is derived from Xeno Kovah's 'Architecture 1001: x86-64 Assembly’ class, available at https://ost.fyi"

SpecialMaths.c

With command line arguments, 2 function parameters, and special maths known to generate particular asm as a teachable moment :)

```c
#include <stdlib.h>
int main(int argc, char ** argv){
    int a;
    //reminder: atoi() converts an
    //ASCII string to an integer
    a = atoi(argv[1]);
    return 2*argc + a;
}
```

```
                        main:
0000000140001000  push rbx
0000000140001002  sub  rsp,20h
0000000140001006  mov  ebx,ecx
0000000140001008  mov  rcx,qword ptr [rdx+8]
000000014000100C  call qword ptr [__imp_atoi (0140004178h)]
0000000140001012  lea  eax,[rax+rbx*2]
0000000140001015  add  rsp,20h
0000000140001019  pop  rbx
000000014000101A  ret
```

SpecialMaths.c

With command line arguments, 2 function parameters, and special maths known to generate particular asm as a teachable moment :)

```c
#include <stdlib.h>
int main(int argc, char ** argv){
    int a;
    //reminder: atoi() converts an
    //ASCII string to an integer
    a = atoi(argv[1]);
    return 2*argc + a;
}
```

```
                        main:
0000000140001000  push rbx
0000000140001002  sub  rsp,20h
0000000140001006  mov  ebx,ecx
0000000140001008  mov  rcx,qword ptr [rdx+8]
000000014000100C  call qword ptr [__imp_atoi (0140004178h)]
0000000140001012  lea  eax,[rax+rbx*2]
0000000140001015  add  rsp,20h
0000000140001019  pop  rbx
000000014000101A  ret
```

“r/mX” Addressing Reminder

- In Intel syntax, ***<u>most of the time</u>*** **square brackets `[]` means to treat the value within as a memory address**, and fetch the value at that address (like dereferencing a pointer)
- An “r/mX” can take 4 forms:
  1. Register → `rbx`
  2. Memory, base-only → `[rbx]`
  3. Memory, base+index*scale → `[rbx+rcx*X]`
     - For X = 1, 2, 4, or 8
  4. Memory, base+index*scale+displacement → `[rbx+rcx*X+Y]`
     - For Y of 1 byte (0-2^8) or 4 bytes (0-2^32)

“r/mX” Addressing Reminder

- In Intel syntax, ***<u>most of the time</u>*** **square brackets `[]` means to treat the value within as a memory address**, and fetch the value at that address (like dereferencing a pointer)
- **THE EXCEPTION THAT PROVES THE RULE!**

3. Memory, base+index*scale → `[rbx+rcx*X]`
   - For X = 1, 2, 4, or 8
4. Memory, base+index*scale+displacement → `[rbx+rcx*X+Y]`
   - For Y of 1 byte (0-2^8) or 4 bytes (0-2^32)



LEA - Load Effective Address

- Uses the *mX* form (really just "m" in the manual) but **<u>is the exception to the rule</u>** that the square brackets `[]` syntax means dereference!
- Frequently used with pointer arithmetic, sometimes for just arithmetic in general
- Example: rbx = 0x2, rdx = 0x1000
  - `lea rax, [rdx+rbx*8+5]`
  - rax = 0x1015, not the value at 0x1015

Book page 112

Setting command line arguments

SpecialMaths Property Pages

Configuration: Debug Platform: Active(x64) Configuration Manager...

- Configuration Properties
  - General
  - Advanced
  - Debugging
  - VC++ Directories
  - C/C++
  - Linker
  - Manifest Tool
  - XML Document Generator
  - Browse Information
  - Build Events
  - Custom Build Step
  - Code Analysis

Debugger to launch: Local Windows Debugger

Command	$(TargetPath)
Command Arguments	**16**
Working Directory	$(ProjectDir)
Attach	No
Debugger Type	Auto
Environment	
Merge Environment	Yes
SQL Debugging	No
Amp Default Accelerator	WARP software accelerator

Command

The debug command to execute.

OK Cancel Apply



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING



STOP

&&

STEP THROUGH

THE ASSEMBLY

&&

CHECK YOUR UNDERSTANDING

SpecialMaths.c takeaways

- When a compiler sees “special math” that can be computed in the form “a + b*X + Y” (derived from the “r/mX” form, where X = {1, 2, 4, 8}, and Y = {0-2^32-1}), then it can compute the result faster if it uses the LEA instruction, rather than a IMUL instruction for instance.
- More evidence that pass-by-register function arguments are being stored onto the stack at some point (this time both ecx and rdx)

```c
#include <stdlib.h>
int main(int argc, char ** argv){
    int a;
    //reminder: atoi() converts an
    //ASCII string to an integer
    a = atoi(argv[1]);
    return 2*argc + a;
}
```

```
main:
push rbx
sub  rsp,20h
mov  ebx,ecx
mov  rcx,qword ptr [rdx+8]
call qword ptr [__imp_atoi (0140004178h)]
lea  eax,[rax+rbx*2]
add  rsp,20h
pop  rbx
ret
```

Instructions we now know (12)

- NOP
- PUSH/POP
- CALL/RET
- MOV
- ADD/SUB
- IMUL
- MOVZX/MOVSX
- **LEA**

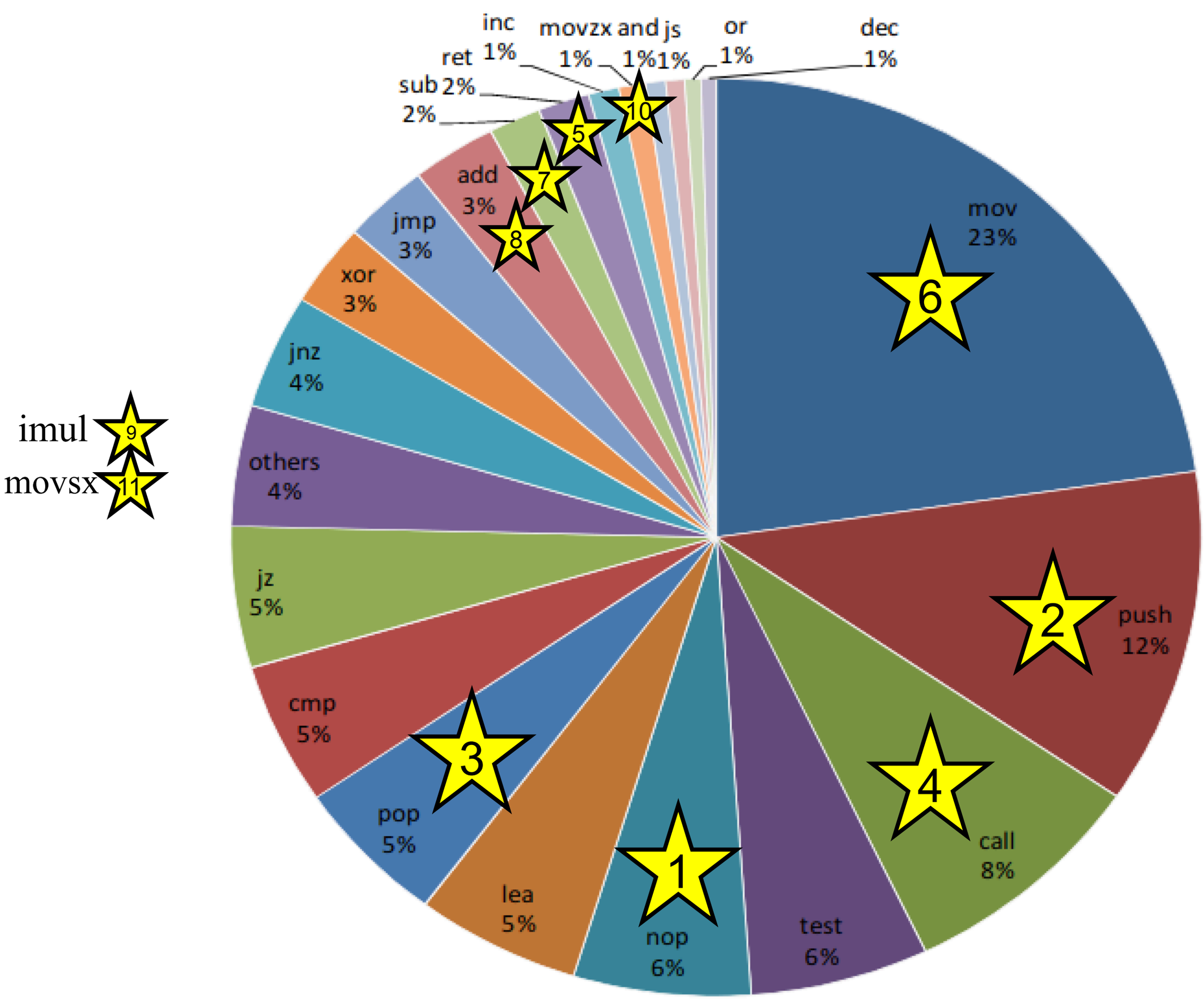


Fig. 2 Web browsers' instruction frequencies

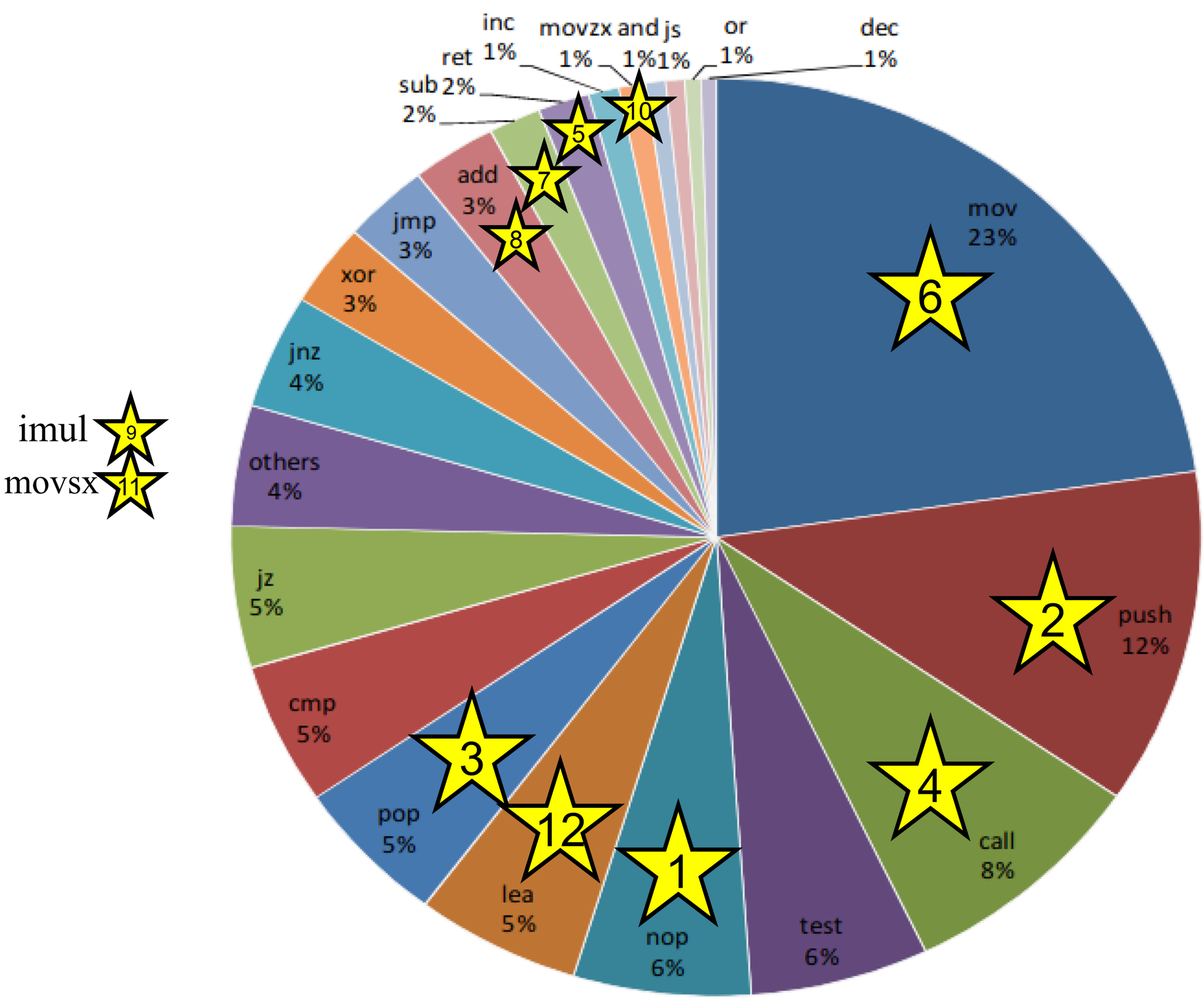


Fig. 2 Web browsers' instruction frequencies




```
0000000140001000  sub          rsp,28h
0000000140001004  lea          rcx,[__NULL_IMPORT_DESCRIPTOR+15C0h (0140006000h)]
000000014000100B  call         printf (0140001080h)
0000000140001010  mov          eax,1234h
0000000140001015  add          rsp,28h
0000000140001019  ret
```

You should be able to read & understand
Hello World's assembly now!

Windows Visual Studio 2019 Community
/GS (buffer overflow protection) option turned off
Disassembled with Visual C++